

동적 계획법 확장판 학습서

DP 마스터북

이 문서는 DP(Dynamic Programming)를 처음 배우는 단계부터, 알고리즘 문제에서 자주 나오는 대표 유형들을 구분하고 점화식을 세우는 감각까지 잡을 수 있게 만든 확장판 정리입니다. 단순 정의뿐 아니라 "어떻게 생각해야 하는지"를 최대한 많이 담았습니다.

1. DP를 가장 짧게 설명하면

DP는 큰 문제를 작은 문제로 나누고, 그 작은 문제의 답을 저장해두면서 재사용하는 방법입니다.

핵심 문장: 같은 계산을 여러 번 하지 않기 위해 답을 저장한다.

2. 이름 때문에 헷갈리지 않기

Dynamic Programming이라는 이름 때문에 뭔가 실시간으로 바뀌는 느낌이 들 수 있지만, 실제 핵심은 "단계적으로 문제를 풀면서 저장한다"는 데 있습니다.

- **Dynamic** : 문제를 단계별로 풀어간다는 느낌
- **Programming** : 여기서는 코딩이 아니라 계획 수립에 가까운 뜻

3. 왜 DP가 필요한가

3-1. 중복 계산 제거

재귀만 쓰면 같은 문제를 여러 번 다시 계산하는 경우가 많습니다. DP는 이 낭비를 줄입니다.

3-2. 최적화 문제 해결

최대값, 최소값, 경우의 수, 가능 여부 같은 문제에 자주 쓰입니다.

3-3. 문제를 구조적으로 풀게 해줌

DP는 무작정 구현하는 것이 아니라 상태와 관계를 정리하는 훈련이 됩니다.

4. DP가 가능하려면 필요한 두 조건

4-1. 최적 부분 구조

큰 문제의 답이 작은 문제의 답으로부터 만들어질 수 있어야 합니다.

4-2. 중복되는 부분 문제

같은 작은 문제를 여러 번 반복 계산하는 구조여야 합니다.

둘 중 하나라도 약하면 DP가 아닌 다른 기법이 더 맞을 수도 있습니다.

5. 가장 기본 예시: 피보나치

5-1. 그냥 재귀

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

이 코드는 `fib(3)`, `fib(2)` 같은 값을 여러 번 다시 계산합니다.

5-2. Bottom-Up DP

```
def fib(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

5-3. 왜 DP인가

- `dp[i]` 에 피보나치 값을 저장합니다.
- 한 번 구한 값은 다시 계산하지 않습니다.
- 이전 두 값만 알면 다음 값을 만들 수 있습니다.

6. Top-Down과 Bottom-Up

구분	Top-Down	Bottom-Up
출발점	큰 문제에서 시작	작은 문제에서 시작
구현	재귀 + 메모이제이션	반복문 + 테이블 채우기
장점	점화식을 그대로 표현하기 쉬움	재귀 깊이 문제 없음
단점	재귀 깊이, 호출 오버헤드	채우는 순서를 직접 잘 잡아야 함

6-1. 왜 재귀는 Top-Down인가

계산이 작은 문제부터 확정되는 느낌이 들어도, 출발 자체가 `stairs(10)` 같은 큰 문제이기 때문에 Top-Down이라고 부릅니다.

7. 메모이제이션과 타블레이션

7-1. 메모이제이션

Top-Down에서 계산한 값을 저장하는 방식입니다.

7-2. 타블레이션

Bottom-Up에서 표를 차례대로 채우는 방식입니다.

8. DP 문제를 풀 때 항상 보는 4단계

1. **dp 정의**: $dp[i]$, $dp[i][j]$ 가 정확히 무엇인지 정한다.
2. **점화식**: 현재 상태를 이전 어떤 상태들로부터 만들지 정한다.
3. **초기값**: 시작 상태를 정한다.
4. **계산 순서**: 어떤 순서로 채워야 이전 값이 먼저 준비되는지 정한다.

9. dp 정의를 잘하는 법

DP에서 가장 중요한 건 **dp** 배열의 의미를 정확히 정하는 것입니다.

좋은 정의 예시

$dp[i]$ = i칸에 도착하는 방법 수

좋지 않은 정의 예시

$dp[i]$ = 그냥 i번째 답

좋은 정의는 "무엇을 구하는지"가 바로 보입니다.

10. 점화식을 세우는 질문

- 현재 상태에 오려면 직전에 어떤 상태들이 가능할까?
- 현재 상태에서 선택지가 몇 가지일까?
- 최댓값/최솟값/경우의 수/가능 여부 중 무엇을 구할까?
- 중복되는 작은 문제가 정말 생기나?

11. 초기값이 왜 중요한가

점화식은 시작점이 없으면 돌아가지 않습니다.

$dp[1] = 1$
 $dp[2] = 2$

이 두 줄이 없으면 계단 오르기 점화식은 시작할 수 없습니다.

12. 계산 순서가 왜 중요한가

예를 들어 $dp[i]$ 가 $dp[i-1]$, $dp[i-2]$ 에 의존하면 당연히 작은 인덱스부터 채워야 합니다.

2차원 DP는 행 우선, 열 우선, 대각선 순서처럼 문제에 맞는 순서를 잡아야 합니다.

13. 대표 예제 1: 계단 오르기

한 번에 1칸 또는 2칸 올라갈 수 있을 때 n 칸에 도착하는 방법 수를 구합니다.

dp 정의

$dp[i]$ = i 칸에 도착하는 방법 수

점화식

$dp[i] = dp[i-1] + dp[i-2]$

초기값

$dp[1] = 1$, $dp[2] = 2$

```
def stairs(n):
    if n == 1:
        return 1

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

이 문제는 구조가 피보나치와 매우 비슷한 전형적인 입문 DP입니다.

14. 대표 예제 2: 1로 만들기

정수 x 를 1로 만드는 최소 연산 횟수를 구합니다.

dp 정의

$dp[i]$ = i 를 1로 만드는 최소 연산 횟수

```
def make_one(x):
    dp = [0] * (x + 1)

    for i in range(2, x + 1):
        dp[i] = dp[i - 1] + 1

        if i % 2 == 0:
            dp[i] = min(dp[i], dp[i // 2] + 1)
        if i % 3 == 0:
            dp[i] = min(dp[i], dp[i // 3] + 1)

    return dp[x]
```

15. 대표 예제 3: 동전 문제

15-1. 최소 동전 개수

$dp[x]$ = 금액 x 를 만드는 최소 동전 개수

```
def min_coins(coins, target):
    INF = 10**9
    dp = [INF] * (target + 1)
    dp[0] = 0

    for coin in coins:
        for x in range(coin, target + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)

    return dp[target] if dp[target] != INF else -1
```

15-2. 경우의 수 세기

$dp[x]$ = 금액 x 를 만드는 방법 수

```
def count_ways(coins, target):
    dp = [0] * (target + 1)
    dp[0] = 1

    for coin in coins:
        for x in range(coin, target + 1):
            dp[x] += dp[x - coin]

    return dp[target]
```

같은 동전 문제라도 **최솟값**과 **경우의 수**는 dp 의미와 식이 완전히 달라집니다.

16. 대표 예제 4: 배낭 문제

각 물건의 무게와 가치가 주어질 때 제한 무게 안에서 가치 합을 최대로 만듭니다.

2차원 정의

$dp[i][w]$ = i 번째 물건까지 고려했을 때 무게 w 이하의 최대 가치

```
def knapsack(items, K):
    n = len(items)
    dp = [[0] * (K + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        weight, value = items[i - 1]
        for w in range(K + 1):
            dp[i][w] = dp[i - 1][w]
            if w >= weight:
                dp[i][w] = max(dp[i][w], dp[i - 1][w - weight] + value)

    return dp[n][K]
```

1차원 최적화

```
def knapsack_1d(items, K):
    dp = [0] * (K + 1)

    for weight, value in items:
        for w in range(K, weight - 1, -1):
            dp[w] = max(dp[w], dp[w - weight] + value)

    return dp[K]
```

0/1 배낭에서는 뒤에서 앞으로 돌아야 같은 물건을 중복 사용하지 않습니다.

17. 대표 예제 5: LIS

최장 증가 부분 수열(Longest Increasing Subsequence)을 구하는 문제입니다.

$dp[i]$ = i 번째 원소를 마지막으로 하는 LIS 길이

```
def lis(arr):
    n = len(arr)
    dp = [1] * n

    for i in range(n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

LIS는 "현재 원소를 끝으로 할 때 이전에 붙일 수 있는 가장 좋은 길이"를 찾는 대표적인 점화식 연습 문제입니다.

18. 대표 예제 6: LCS

두 문자열의 최장 공통 부분 수열(Longest Common Subsequence)을 구합니다.

$dp[i][j]$ = 첫 번째 문자열 앞 i 글자와 두 번째 문자열 앞 j 글자의 LCS 길이

```
def lcs(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[n][m]
```

19. 경우의 수 DP, 최적화 DP, 판별 DP

경우의 수

몇 가지 방법이 있는가

최솟값/최댓값

최소 비용, 최대 점수

가능 여부

가능/불가능, True/False

문제를 읽을 때 먼저 이 셋 중 어디에 속하는지 보는 습관이 좋습니다.

20. 1차원 DP와 2차원 DP를 구분하는 감각

1차원 DP

- 한 개의 상태만 추적하면 될 때
- 예: 위치, 금액, 길이

2차원 DP

- 두 개의 상태가 동시에 필요할 때
- 예: 문자열 2개, 아이템 번호와 무게

21. 구간 DP란 무엇인가

`dp[i][j]` 가 구간 `[i, j]` 에 대한 답을 의미하는 유형입니다.

- 행렬 곱셈 순서
- 팰린드롬 관련 문제
- 파일 합치기

보통 길이가 짧은 구간부터 긴 구간으로 확장하며 계산합니다.

22. 트리 DP란 무엇인가

트리의 각 정점을 기준으로 서브트리 정보를 모아서 답을 구하는 방식입니다.

- `dp[node][0]` : 이 정점을 선택하지 않았을 때
- `dp[node][1]` : 이 정점을 선택했을 때

사회망 서비스, 독립집합 같은 문제에 자주 나옵니다.

23. 비트마스크 DP란 무엇인가

방문한 상태를 비트마스크로 표현하는 DP입니다.

- 상태 수는 많지만 정점 수가 작을 때 유용
- 대표 문제: 외판원 순회(TSP)

`dp[mask][i]` = mask 상태를 방문했고 현재 i에 있을 때의 최소 비용

24. DAG 위의 DP

사이클이 없는 방향 그래프에서는 위상정렬 순서대로 DP를 할 수 있습니다.

- 선행 관계가 있는 작업 순서
- 경로 개수 세기
- 최장 경로 문제

25. DP와 그리디 차이

기법	핵심
그리디	지금 당장 제일 좋아 보이는 선택
DP	가능한 이전 상태들을 모두 고려해 누적 최적해 구성

그리디는 항상 정답이 보장되지 않지만, DP는 점화식이 맞으면 최적해를 보장하는 경우가 많습니다.

26. DP와 BFS/DFS 차이

BFS/DFS는 상태 공간을 탐색하는 느낌이고, DP는 상태들 사이의 점화 관계를 이용해 누적 계산하는 느낌입니다.

물론 DAG DP처럼 그래프 탐색과 DP가 섞이는 경우도 많습니다.

27. 공간 최적화

모든 이전 상태가 필요한 게 아니라 몇 개만 필요하면 배열 대신 변수만 써도 됩니다.

```
def fib(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

배낭 문제처럼 2차원 DP도 1차원으로 줄일 수 있는 경우가 많습니다.

28. 자주 하는 실수 모음

- dp 정의가 불명확함

- 점화식을 틀리게 세움
- 초기값을 빼먹음
- 정답 위치를 틀리게 출력함
- 인덱스 0부터인지 1부터인지 헷갈림
- 계산 순서를 잘못 정함
- 메모리 크기를 너무 크게 잡음

29. DP 문제를 보면 던질 질문

1. 이 문제는 최댓값, 최솟값, 경우의 수, 가능 여부 중 무엇인가?
2. 현재 상태를 무엇으로 표현할 수 있을까?
3. 현재 상태는 이전 어떤 상태들로부터 오는가?
4. 작은 문제 답을 저장하면 도움이 되는가?
5. 1차원으로 되는가, 2차원이 필요한가?
6. 계산 순서를 어떻게 정해야 하는가?

30. DP 문제를 처음 공부할 때 추천 순서

1. 피보나치
2. 계단 오르기
3. 1로 만들기
4. 동전 1, 동전 2
5. 평범한 배낭
6. LIS
7. LCS
8. 구간 DP
9. 트리 DP
10. 비트마스크 DP

31. 백준에서 DP 감각을 키우는 팁

- 문제를 보자마자 "DP인가?"보다 "작은 문제를 정의할 수 있나?"를 먼저 생각합니다.
- 점화식을 글로 먼저 써보고 코드로 옮깁니다.
- 작은 예시를 손으로 채워보며 규칙을 확인합니다.
- 틀렸을 때는 점화식보다 `dp` 정의와 초기값부터 다시 봅니다.

32. 실제 풀이 템플릿

```
# 1. dp 정의
# dp[i] = ...

# 2. 배열 준비
dp = ...

# 3. 초기값
dp[0] = ...

# 4. 점화식으로 채우기
for i in range(...):
    dp[i] = ...

# 5. 정답 출력
print(dp[...])
```

33. 마무리 압축 정리

DP의 본질

작은 문제의 답을 저장해 큰 문제를 빠르게 푼다

제일 중요한 것

dp 정의와 점화식

가장 흔한 실수

정의를 흐리거나 초기값이 틀리는 것

공부법

쉬운 1차원 문제부터 차근차근 올라가기

34. 마지막 한 문장

DP는 코드를 외우는 기술이 아니라, 문제를 상태로 바꿔서 생각하는 기술입니다.